

프로젝트 개요

프로젝트명

온누리상품권 가맹점 검색 & 분석 챗봇

프로젝트 기간

2024.12.22 (월) ~ 2024.12.31 (수) (6일)

프로젝트유형 개인 프로젝트

프로젝트 목표

1. 핵심 목표

- 온누리상품권 가맹점 정보를 자연어로 검색할 수 있는 대화형 챗봇 개발
- RAG(Retrieval-Augmented Generation) 기반 정보 검색 시스템 구축
- 공공데이터를 활용한 실용적인 서비스 제공

2. 기술적 목표

- 대규모 데이터(15만+ 레코드) 전처리 및 파이프라인 구축
- 벡터 데이터베이스를 활용한 의미 기반 검색 구현
- LLM과 구조화 데이터 검색의 하이브리드 시스템 설계

3. 학습 목표

- LangChain을 활용한 RAG 시스템 실무 경험
- 공공데이터 전처리 및 활용 능력 향상
- 데이터 엔지니어링 역량 강화

프로젝트 배경 및 필요성

문제 정의

- 정보 접근성: 온누리상품권 가맹점 정보가 분산되어 있어 찾기 어려움
- 검색의 한계: 단순 키워드 검색으로는 사용자 의도 파악 어려움
- 데이터 활용: 15만+ 가맹점 데이터와 지역별 통계 데이터가 유기적으로 연결되지 않음

기대 효과

- 사용자: 자연어로 쉽게 가맹점 검색 가능
- 소상공인: 가맹점 정보 노출 증가
- 정책 분석: 지역별 판매/회수 현황 분석을 통한 인사이트 제공

데이터 소개

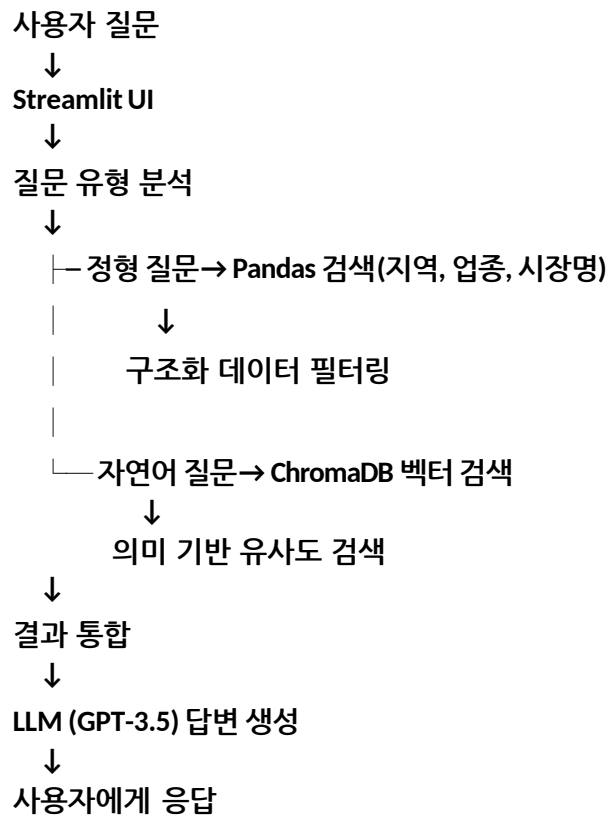
사용 데이터

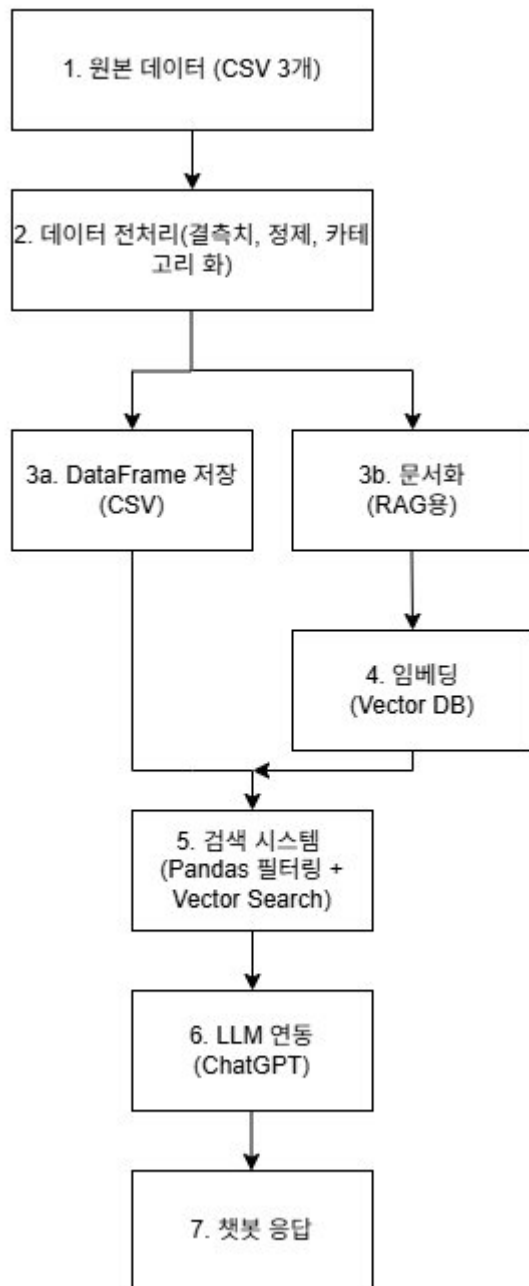
데이터명	출처	규모	용도
전국 온누리상품권 가맹점 현황	소상공인시장진흥공단	150,541건	가맹점 검색 (메인)
온누리상품권 지역별 회수 현황	소상공인시장진흥공단	17개 지역	사용량 통계 분석
온누리상품권 지역별 판매 현황	소상공인시장진흥공단	17개 지역	판매량 통계 분석

주요 기능

- A. 지역 기반 검색
- B. 지역 + 업종 조합
- C. 요약·통계(비율)질문
- 등등

시스템 아키텍처(전체 구조, 데이터 파이프라인)





기술 스택

- Python 3.11+
- LangChain (RAG 구현)
- OpenAI API (GPT-3.5-turbo)
- ChromaDB (Vector DB)
- Pandas (데이터 처리)
- Streamlit (UI)

개발 일정

DAY 1 (12/22, 월): 기획 + 환경 설정

오전 :

- 요구사항 정리
- 필수 4개 질문-답변 시나리오 작성
- 추가 기능 우선순위 정리

오후 :

- 개발 환경 세팅 (Python, LangChain, Streamlit)
- 데이터 다운로드 및 확인
- GitHub repo 생성

저녁 :

- 기술 스택 최종 결정
- 프로젝트 구조 설계

DAY 2 (12/23, 화): 데이터 전처리 + 저장

오전 :

- CSV 데이터 로드 및 확인
- 결측치 처리 (가맹점명 40개, 취급품목 293개)
- 데이터 정제 (공백 제거, 형식 통일)
- 정제된 DataFrame 저장

오후 :

- 데이터1 (가맹점 15만개) 적재
- 데이터2 (지역별 통계) 적재
- 지역검색 함수, 업종검색 함수, 복합 검색 함수, 통계

DAY 3 (12/24, 수): RAG 시스템 구축

오전 :

- 가맹점 정보 문서화 (가맹점명 + 소재지 + 취급품목)
- 텍스트 전처리 및 청킹

오후 :

- 임베딩 모델 선택 (OpenAI)
- Vector DB 구축 (ChromaDB)
- 15만개 가맹점 임베딩 생성 및 저장

녁 :

- 검색 테스트 ("경기도 음식점" 검색)
- ✓체크포인트: RAG 기본 작동 확인

DAY 4 (12/26, 금): 챗봇 로직 구현 *

오전 :

- LLM API 연동 (OpenAI GPT-3.5/4)
- 프롬프트 엔지니어링 (시스템 프롬프트 작성)

오후 :

- 필수 4개 질문 처리 로직 구현
- 지역 필터 질문 처리
- 취급품목 필터 질문 처리
- 디지털형 필터 질문 처리
- 일반 정보 질문 처리
- Day 4 끝나고 필수 기능 완성했는데 시간 남으면:
- Pandas → SQLite 마이그레이션
- 간단한 인덱싱 추가
- SQL 쿼리로 변환

저녁 :

- 각 질문 테스트 및 디버깅
- ✓핵심 체크포인트: 필수 4개 완성 ✓

DAY 5 (12/27, 토): UI 구현 + 통합

오전 :

- Streamlit 기본 레이아웃 구현
- 채팅 인터페이스 UI
- 입력창 + 대화 히스토리

오후 :

- 백엔드(RAG + LLM) ↔ 프론트엔드(Streamlit) 연결
- 전체 플로우 통합 테스트
- 필수 4개 질문 실제 UI에서 테스트

저녁 :

- 시간 여유 있으면:
- 질문 5번 추가 (지역별 가맹점 수 순위)
- 간단한 차트 추가 (matplotlib)
- 시간 부족하면:
- UI 디버깅만

DAY 6 (12/28, 일): 마무리 + 문서화

오전 :

- 전체 테스트 (다양한 질문 입력)
- 버그 수정, 엡지 케이스 처리
- 에러 메시지 개선

오후 :

- 시간 여유 있으면:
- 질문 6번 추가 (2024년 회수금액 순위 - 데이터2)
- 질문 7번 추가 (지역 비교)
- 필수:
- README.md 작성
- 프로젝트 소개
- 설치 방법
- 실행 방법
- 주요 기능
- 기술 스택
- 시연 영상 녹화 또는 스크린

샷

저녁 :

- 코드 정리 및 주석 추가
- GitHub 푸시
 - 코드 정리 및 주석 추가
 - GitHub 푸시

프로젝트 구조

```
chatbot_project/
├── .vscode/                # VS Code 설정 파일 (디버그, 워크스페이스 설정 등)
│
├── docs/                  # 프로젝트 문서 (기획서, 설계 문서, 정리 노트 등)
│
├── notebooks/             # EDA, 전처리, 실험용 Jupyter Notebook
│                           # (ipynb 단계에서 검증 후 src 코드로 이전)
│   ├── 01_data_exploration.ipynb    # 기본통계량 확인
│   └── 02_preprocessing.ipynb        # 전처리 및 정제된 데이터 생성+테스트
├── page/                  # Streamlit 멀티페이지 UI 구성 디렉터리
│   ├── __pycache__/          # Python 캐시 파일 (자동 생성)
│   ├── __init__.py           # page 패키지 인식용
│   ├── intro.pswp            # 임시 파일 (편집기 백업 파일로 보임)
│   ├── intro.py              # 서비스 소개 / 메인 설명 페이지
│   ├── project1.py           # 프로젝트 1 페이지 (Streamlit UI)
│   ├── project2.py           # 프로젝트 2 페이지 (Data flow)
│   └── project3.py           # 프로젝트 3 페이지 (챗봇 페이지)
│
└── src/                   # 핵심 비즈니스 로직 (UI와 분리)
```

```

|   |─ __pycache__/          # Python 캐시 파일
|   |─ utils/                # 공통 유틸 함수 및 상수
|   |   |─ __pycache__/      # Python 캐시 파일 (자동 생성)
|   |   |─ project2_desc.py.pswp    # 임시 파일 (편집기 백업 파일로 보임)
|   |   |─ project1_desc.py        # 프로젝트 1 페이지 설명글
|   |   |─ project2_desc.py        # 프로젝트 2 페이지 설명글
|   |   |─ project3_desc.py        # 프로젝트 3 페이지 설명글
|   |   └─ __init__.py            # utils 패키지 인식용
|   |
|   |─ __init__.py            # src 패키지 인식용
|   └─ chatbot.py              # 사용자 질의 처리 메인 로직
|       (LLM 호출, 프롬프트 구성, 응답 생성)
|   └─ rag_setup.py            # RAG 파이프라인 구성
|       (임베딩, 벡터DB 로딩, Retriever 설정)
|   └─ search_engine.py        # 검색 로직
|       (키워드 검색, 벡터 검색, 하이브리드 검색)
|
|─ vectordb/                  # 벡터 데이터베이스 저장 디렉터리
|   (FAISS / Chroma / 기타 로컬 벡터 저장소)
|
|─ .env                       # 환경 변수 파일 (API Key, 경로 등)
|
|─ .gitignore                 # Git 추적 제외 파일 목록
|
|─ api_keys.txt               # API 키 보관 파일 (실서비스에서는 사용 비권장)
|
|─ app.py                     # Streamlit 앱 메인 엔트리 포인트
|   # (단일 페이지 또는 기본 실행용)
|
|─ app_multipage.py           # Streamlit 멀티페이지 실행 진입점
|   # page 디렉터리와 연결
|
|─ multipage.py               # 멀티페이지 라4팅 / 페이지 관리 로직
|
|─ area_onnuri.csv            # 온누리상품권 원본 데이터
|
|─ cleaned_onnuri.csv         # 전처리 완료된 온누리 데이터
|
|─ README.md                  # 프로젝트 개요, 구조 설명, 실행 방법
|

```

└─ requirements.txt

Python 패키지 의존성 목록

제약사항 및 한계

1. 데이터 한계

- 소재지 정보: 시/도 단위만 제공 (구/군 정보 없음)
- 대안: 시장명 기반 검색 제공, 카카오 API기반 구단위 검색

2. 검색 범위

- 지원: 시/도 단위, 업종, 시장명
- 미지원: 구/군 단위(예: "서울 강남구")

향후 개선 방안

입력과 출력 데이터 출처와 어떻게 출력했는지 적기

1. 상세 주소 데이터 확보 → 구/군 단위 검색
2. 실시간 영업시간 정보 추가
3. 리뷰/평점 데이터 통합
4. 모바일 앱 개발
5. 음성 인터페이스 추가
6. 개인화 추천 시스템
7. PostgreSQL 마이그레이션 (확장성)

학습 및 성장

기술 역량

- 공공데이터 전처리 경험
- RAG 시스템 설계 및 구현
- 벡터 데이터베이스 활용
- LLM 프롬프트 엔지니어링
- 데이터 파이프라인 구축

프로젝트 관리

- 기획안 작성
- 일정 관리 (6일 단위)
- Git을 활용한 버전 관리
- 기술 문서 작성

포트폴리오

- 데이터 엔지니어 역량 강화
- 실무 수준의 프로젝트 경험
- 협업 가능한 코드 작성 능력

참고 자료

공공데이터

- [공공데이터포털](#)

기술 문서

- [LangChain Documentation](#)
- [ChromaDB Documentation](#)
- [OpenAI API Reference](#)
- [Streamlit Documentation](#)